BİL377 - HAFTA 08

Dinamik Programlama Uygulamaları

Minimum para sayısı, 0/1 sırt çantası ve seçimi geri kurma

Durum, seçenek ve en iyi sonuç

Fibonacci ile bu haftanın bağlantısı

Hazır alt sonuçları kullanma fikri aynı, durumun anlamı değişiyor.

Fibonacci: önceki iki sonucun toplamı

Para üstü: adayların en küçük para adedi

Çanta: alma ve almama adaylarının en büyük değeri

Her hücre belirli bir sorunun cevabıdır

Önce hücrenin anlamı, sonra hücreyi hesaplayan ifade

Minimum para sayısı problemi

Birimler [1,3,4], hedef tutar 6. Her birimden yeterince para var.

İstenen: toplamı tam 6 olan en az para adedi

4 + 1 + 1 = 6, toplam 3 para

3 + 3 = 6, toplam 2 para

Çıktı: adet 2, kullanılan paralar [3,3]

Paraların toplam değeri ile para adedi farklı büyüklüklerdir.

En büyük parayı seçmenin sınırı

Her adımda kalana sığan en büyük birimi seçen açgözlü yöntem.

6 için ilk seçim 4, kalan tutar 2

Kalan 2 için 1 ve 1 gerekir

Açgözlü sonuç 3 para, optimum sonuç 2 para

Bir seçim, kalan alt problemi değiştirebilir

Bu birim sisteminde açgözlü yöntem optimumu garanti etmez.

Para üstünde durumun anlamı

M(a): tam a tutarını oluşturmak için gereken en az para adedi.

M(0)=0: sıfır ödeme için para gerekmez

M(3)=1: tek 3 birimi yeterlidir

M(6)=2: iki adet 3 birimi yeterlidir

Ulaşılamayan tutarın değeri ∞

M(a), tutarı değil o tutarı oluşturan en az adedi saklar.

Son paraya göre adaylar

Son kullanılan para p ise önce a–p tutarı oluşturulmuştur.

Son para 1: 1 + M(5)

Son para 3: 1 + M(3)

Son para 4: 1 + M(2)

M(6)=min(3,2,3)=2

Eklenen 1, seçilen paranın adedidir. Kalan tutar p kadar düşer.

Minimum para bağıntısı

P pozitif para birimleri kümesi, a pozitif hedef tutardır.

$$M(0)=0$$

$$M(a)=\min \{1+M(a-p): p\in P, p\leq a\}$$

Uygun veya erişilebilir aday yoksa $M(a)=\infty$

Hesap sırası: $a=1,2,\ldots,A$

p pozitif olduğundan her alt durum daha küçük bir tutardır.

6 tutarı için bütün küçük cevaplar

Birimler [1,3,4]. Her satırda yalnız uygun son paralar karşılaştırılır.

a	Aday adetler	M(a)	Seçilen son para
1	1+M(0)=1	1	1
2	1+M(1)=2	2	1
3	1+M(2)=3, 1+M(0)=1	1	3
4	2, 2, 1	1	4
5	2, 3, 2	2	1
6	3, 2, 3	2	3

M dizisi: [0,1,2,1,1,2,2]

Para tablosunun Python kodu


```
dp = [0] + [inf] * hedef  # Adetler
son = [-1] * (hedef+1)     # Son seçim
for a in range(1, hedef+1):
    for p in paralar:
        if p <= a:
            aday = 1 + dp[a-p]
            if aday < dp[a]:
                dp[a] = aday
                son[a] = p
```

dp ve son

dp[a]: en az adet.

son[a]: seçilen son para.

Sıkı < eşitlikte eski seçimi korur.

Tam fonksiyon: kod/ders_ornekleri.py içindeki para_tablosu

M(6) hücresinin güncellenmesi

Dış döngü a=6 değerine ulaştığında M(2), M(3), M(5) hazırdır.

Başlangıç: dp[6]=∞, son[6]=−1

p=1: 1+dp[5]=3, kayıt (3,1)

p=3: 1+dp[3]=2, kayıt (2,3)

p=4: 1+dp[2]=3, kayıt değişmez

Sonuç: dp[6]=2, son[6]=3

Tablodan kullanılan paraları çıkarma

Başlangıç tutarı 6. son[a], kalan hesabın son seçimini saklar.

Kalan a	son[a]	Listeye eklenen	Yeni kalan
6	3	3	3
3	3	3	0

Kullanılan paralar [3,3], toplam 6 ve adet 2

Para seçimini geri kuran kod


```
if dp[hedef] == inf:
    return None  # Tam ödeme yok
kullanilan, a = [], hedef
while a > 0:
    p = son[a]       # Son seçimi oku
    kullanilan.append(p)
    a -= p           # Kalanı azalt
return int(dp[hedef]), kullanilan
```

Bitiş koşulu

Her p pozitif.

Kalan tutar kesin azalır.

0 tutarda boş liste döner.

min_coins([1,3,4],6) sonucu (2,[3,3])

Çözümü olmayan tutar

Birimler [4,6], hedef 5. Her birimden sınırsız sayıda var.

$M(0)=0$

$M(1),M(2),M(3)=\infty$

$M(4)=1$, fakat $M(5)=1+M(1)=\infty$

Fonksiyon sonucu None

Erişilemez hedef ile sıfır adetlik ödeme farklı sonuçlardır.

Özyinelemeli para hesabı


```
def f(a):
    if a == 0:
        return 0
    en_iyi = inf
    for p in paralar:
        if p <= a:
            aday = 1 + f(a-p)
            en_iyi = min(en_iyi, aday)
    return en_iyi
```

Kalan tutar durumu

f(6), f(5), f(3), f(2) ister.

f(5) de f(2) ister.

Aynı kalan tekrar hesaplanır.

Naif sürüm bağıntıyı doğrudan yürütür.

Önbellekli para hesabı


```
from functools import lru_cache

@lru_cache(maxsize=None)
def f(a):
    if a == 0:
        return 0
    return min(
        (1+f(a-p) for p in paralar
         if p <= a), default=inf)
```

Önbellek anahtarı a

Aynı a tekrar gelirse hazır adet.

Her üst çağrıda yeni f tanımı.

Özyineleme yığını devam eder.

Para listesi bu çağrı boyunca sabittir.

Para hesabındaki kesin sayımlar

Birimler [1,3,4], hedef 6. İstek, fonksiyona her girişi sayar.

Yöntem	İstek	Yeni hesap	Uygun aday
Naif	24	24	23
Memo	14	7	13
Tablo	Özyineleme yok	7	13

Tabloda 18 birim kontrolünün 13 tanesi tutara uygundur.

Para üstünde zaman ve bellek

k farklı birim, hedef A. Analiz, hazır birim listesinden sonra başlar.

Tablo: A tutar × k birim, Θ(Ak) zaman

dp ve son: 2(A+1) hücre, Θ(A) alan

Memo: O(Ak) zaman, O(A) kayıt ve yığın

Naif: kaba üst sınır O((k+1)^(A+1))

k≥2 için naif kaba üst sınır O(k^A). Her para en az 1 birimdir.

Sekiz kilogramlık ekipman çantası

Dört nesnenin her birinden yalnızca bir tane var. Kapasite 8 kg.

A: 2 kg, değer 8

B: 3 kg, değer 14

C: 4 kg, değer 16

D: 5 kg, değer 20

Amaç: toplam ağırlık ≤8 iken toplam değeri en yüksek yapmak.

Geçerli çanta seçimleri

Ağırlık sınırı 8. Her nesne en fazla bir kez kullanılabilir.

Seçim	Ağırlık	Değer	Durum
A+B	5	22	Sığar
A+D	7	28	Sığar
B+C	7	30	Sığar
B+D	8	34	Sığar
C+D	9	36	Sığmaz
A+B+C	9	38	Sığmaz

Bu örnekte en iyi seçim B+D, toplam değer 34.

Çantada iki indeks neden gerekir?

K(i,c): ilk i nesneden, en çok c kg ile elde edilen en yüksek değer.

K(1,3)=8: yalnız A kullanılabilir

K(2,3)=14: A veya B kullanılabilir

Kapasite aynı, kullanılabilir nesneler farklı

Durum anahtarı (i,c) çiftidir

Yalnız kalan kapasite, 0/1 modelinin bütün bilgisini taşımaz.

Çanta tablosunun temel durumları

Satır i izin verilen ilk i nesneyi, sütun c kapasite sınırını gösterir.

K(0,c)=0: kullanılabilecek nesne yok

K(i,0)=0: pozitif ağırlıklı nesne sığmaz

c=0,1,…,8 sütunları toplam 9 adettir

i=0,1,…,4 satırları toplam 5 adettir

Dört nesne ve kapasite 8 için 5×9=45 hücre

Bir nesne için alma ve almama

i. nesnenin ağırlığı w, değeri v. Son karar bu nesne üzerinedir.

Almama adayı: $K(i-1,c)$

Alma adayı: $v + K(i-1,c-w)$, $w \leq c$ ise

$K(i,c)$: bu iki adayın en büyüğü

w>c ise yalnız almama adayı geçerlidir

Her iki aday da i–1 satırını okur. Böylece nesne tekrar kullanılamaz.

B nesnesi eklenirken K(2,5)

B: ağırlık 3, değer 14. Önceki satır yalnız A nesnesini içerir.

Aday	Okunan hücre	Hesap	Sonuç
B alınmaz	K(1,5)=8	8	8
B alınır	K(1,2)=8	14+8	22

K(2,5)=22, seçim A+B, toplam ağırlık 5

A ve B için tablonun ilk satırları

Sütun başlıkları kapasite, hücre değerleri en yüksek puandır.

Nesneler	0	1	2	3	4	5	6	7	8
Yok	0	0	0	0	0	0	0	0	0
A	0	0	8	8	8	8	8	8	8
A,B	0	0	8	14	14	22	22	22	22

B için 3 ve 4 kapasitede 14, 5 ve üstünde 22 elde edilir.

C eklenirken kapasiteye göre seçim

C: ağırlık 4, değer 16. Yalnız A ve B satırı okunur.

c	C alınmaz	C alınır	Yeni en iyi
4	14	16+0	16
5	22	16+0	22
6	22	16+8	24
7	22	16+14	30
8	22	16+14	30

Yeni nesne her kapasitede seçilmek zorunda değildir.

D ile son hücrenin hesabı

D: 5 kg, değer 20. Hedef hücre K(4,8).

Karar	Alt problem	Toplam değer
D alınmaz	K(3,8)=30	30
D alınır	K(3,3)=14	20+14=34

K(4,8)=34. D ile birlikte kalan 3 kg kapasite B nesnesine ayrılır.

0/1 çantasının tamamlanmış tablosu

Her satır yeni bir nesneye izin verir. K(4,8)=34.

Nesneler	0	1	2	3	4	5	6	7	8
Yok	0	0	0	0	0	0	0	0	0
A	0	0	8	8	8	8	8	8	8
A,B	0	0	8	14	14	22	22	22	22
A,B,C	0	0	8	14	16	22	24	30	30
A,B,C,D	0	0	8	14	16	22	24	30	34

Satır 4, sütun 8: B+D seçiminin değeri 34

Çanta tablosunun Python kodu


```
dp = [[0]*(C+1) for _ in range(n+1)]
for i in range(1, n+1):
    w, v = agirliklar[i-1], degerler[i-1]
    for c in range(C+1):
        dp[i][c] = dp[i-1][c]  # Almama
        if w <= c:
            aday = v + dp[i-1][c-w]
            dp[i][c] = max(dp[i][c], aday)
```

i ve i–1 ayrımı

Tablo satırı i: ilk i nesne.

Python nesne indeksi i–1.

Adaylar önceki satırdan.

Bağımsız satırlar liste üretimiyle oluşturulur.

Çantanın özyinelemeli bağıntısı


```
def f(i, c):
    if i == 0:
        return 0
    alma = f(i-1, c)
    if agirliklar[i-1] > c:
        return alma
    al = degerler[i-1] + f(
        i-1, c-agirliklar[i-1])
    return max(alma, al)
```

Her dal i değerini azaltır

Sığmayan nesne atlanır.

İki dalda da i−1 kullanılır.

En fazla n karar derinliği.

c aynı kalsa bile i azaldığından özyineleme sonlanır.

Değer tablosundan nesneleri çıkarma

Başlangıç (i,c)=(4,8). Eşit değer varsa nesne atlanır.

i,c	Karşılaştırma	Karar	Sonraki durum
4,8	34 ≠ 30	D alınır	3,3
3,3	14 = 14	C atlanır	2,3
2,3	14 ≠ 8	B alınır	1,0
1,0	0 = 0	A atlanır	0,0

Seçilen nesneler B,D. Sıfır tabanlı indeksler [1,3].

Çanta seçimini geri kuran kod


```
secilen, c = [], kapasite
for i in range(n, 0, -1):
    if dp[i][c] != dp[i-1][c]:
        secilen.append(i-1)
        c -= agirliklar[i-1]
secilen.reverse()
return dp[n][kapasite], secilen
```

Kayıt olarak tam tablo

Satır farkı: nesne alınmış.

Eşitlik: nesne atlanabilir.

Ağırlık yalnız alınca düşer.

knapsack([2,3,4,5],[8,14,16,20],8) sonucu (34,[1,3])

Çanta tablosunun doğruluk gerekçesi

K(i,c), izin verilen bütün uygun alt kümelerin en iyi değeridir.

Başlangıç: boş nesne kümesinin değeri 0

Son nesneyi içermeyen çözümler: K(i–1,c)

Son nesneyi içeren çözümler: v+K(i–1,c–w)

İki grubun en iyisi bütün çözümlerin en iyisidir

Gruplar son nesnenin alınmasına göre ayrılır ve bütün seçimleri kapsar.

Çantada zaman ve bellek hesabı

n nesne, tamsayı kapasite C. Birim maliyetli sayı modeli.

Tam tablo: $(n+1)(C+1)$ hücre

$n(C+1)$ hücre ziyareti, en çok nC alma adayı

Zaman $\Theta(nC)$, ek alan $\Theta(nC)$, $n, C \geq 1$

Geri kurma $O(n)$, çıktı listesi en çok n indeks

Örnekte 45 hücre, 36 ziyaret ve 22 uygun alma adayı

Sözde polinom zaman ne demektir?

Tablonun boyutu kapasitenin sayısal değerine bağlıdır.

C=8 için her satırda 9 hücre

C=1000000 için her satırda 1000001 hücre

C ikili yazımda yaklaşık $\log_2 C$ bit gerektirir

$\Theta(nC)$, C'nin yazım uzunluğunda polinom değildir

Az bit ile yazılan büyük kapasite, çok büyük bir tablo gerektirebilir.

Çantada tek satırla değer hesabı


```
dp = [0]*(kapasite+1)
for w, v in zip(agirliklar, degerler):
    for c in range(kapasite, w-1, -1):
        dp[c] = max(dp[c], v+dp[c-w])
return dp[kapasite]
```

Kapasite azalan sırada

Okunan dp[c-w] eski satıra aittir.

Alan Θ(C).

Bu sürüm yalnız değeri döndürür.

Örnekte 45 hücre yerine 9 hücre, aynı en iyi değer 34

Döngü yönü nesne kullanımını değiştirir

Tek nesne: ağırlık 2, değer 8. Kapasite 4, 0/1 modeli.

Sıra	İlk işlem	Sonraki işlem	Son değer
Artan	c=2: dp[2]=8	c=4: 8+dp[2]=16	16
Azalan	c=4: 8+0=8	c=2: dp[2]=8	8

Artan sıra aynı nesneyi iki kez kullanabilir. 0/1 için doğru cevap 8.

İki problemin model karşılaştırması

Durum ve tekrar kullanma koşulu, okunacak alt hücreyi belirler.

Özellik	Minimum para	0/1 çanta
Durum	Tam a tutarı	İlk i nesne, en çok c
Amaç	Adedi en aza indirme	Değeri en çoğa çıkarma
Tekrar kullanım	Aynı birim sınırsız	Her nesne en fazla bir
Geçiş	1+M(a−p)	v+K(i−1,c−w)
Ulaşılamayan	∞, çıktı None	Boş seçim her zaman geçerli

Benzer döngüler, aynı problem koşulları anlamına gelmez.

Alıştırma: farklı para sistemi

Birimler [1,4,6], hedef 8. Birimler tekrar kullanılabilir.

En büyük birimle başlayan ödeme kaç para kullanır?

M(4), M(6), M(7), M(8) değerleri nedir?

En az adetli ödeme hangi paralardan oluşur?

son[8] hangi seçimle tutulabilir?

Minimum adet ile toplam ödeme tutarını ayrı ifade etmek gerekir.

Alıştırma: küçük 0/1 çanta

A=(2 kg,3 puan), B=(3 kg,4 puan), C=(4 kg,5 puan). Kapasite 5.

K(1,5) ve K(2,5) kaçtır?

C alınırsa kalan kapasite kaç olur?

K(3,5) için alma ve almama adayları nedir?

Hangi nesneler seçilir, toplam ağırlık ve değer kaçtır?

Kapasiteyi aşan yüksek puanlı seçenekler geçerli çözüm değildir.

Sınır durumları ve sözleşme

Para birimleri ve ağırlıklar pozitif, hedef/kapasite ve değerler negatif olmayan int.

Durum	Beklenen davranış
Para hedefi 0	(0,[])
Boş birimler, hedef 5	None
[4,6], hedef 5	None
Boş çanta veya kapasite 0	(0,[])
0 para, negatif ağırlık, negatif hedef	ValueError
Kesirli sayı veya bool	TypeError

Yanlış girdi ile geçerli ama çözümsüz problem ayrı ele alınır.

Ödev: sınırlı stokla para üstü

Her birim için kullanılabilecek para adedi artık sınırlıdır.

Fonksiyon: sinirli_para(birimler,stoklar,hedef)

Minimum adet ve kullanılan adetler listesi

Yetersiz stokta None sonucu

Örnek: [1,3,4], stok [2,1,1], hedef 6

Örnek optimum 3 para: adetler [2,0,1], yani 1+1+4

Gerçek uygulama: paketleme ve kaynak seçimi

DP, çok sayıda seçeneğin ortak alt hesaplarını tekrar kullanır.

Kargo: ağırlık sınırında yüksek öncelikli paketler

Bütçe: tek kaynak sınırında proje seçimi

İşlem birleştirme: hedefi en az parça ile tamamlama

Sonuç: optimum değerle birlikte uygulanabilir seçim

Model varsayımları: ayrık seçimler, toplamsal değer ve tek kaynak sınırı